LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

"PENGEMBANGAN SISTEM PERBANKAN MINI
MODUL 1–5"



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH PEMOGRAMAN BERORIENTASI OBJEK
DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM.
ASISTEN PRAKTIKUM : EZZA ADDINI

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

A. TAHAPAN PELAKSANAAN PRAKTIKUM

Praktikum Modul 1 sampai Modul 5 dilakukan dengan mengembangkan satu program sistem perbankan mini secara bertahap. Setiap modul menambahkan kemampuan pada program yang sudah dibuat sebelumnya. Class rekening dan menu utama menjadi dasar, kemudian program dilengkapi dengan riwayat transaksi, pengamanan data, jenis rekening, dan proses transfer.

1. Modul 1: Class, Object, Constructor, dan Method

Pengembangan dimulai dengan membuat class Rekening sebagai model untuk menyimpan data rekening. Data yang digunakan meliputi nomor rekening, nama pemilik, dan saldo. *Constructor* mengisi data awal ketika objek rekening dibuat. *Method* setorTunai() digunakan untuk menambah saldo, sedangkan cekInformasi() menampilkan informasi rekening. *Method* tarikTunai() kemudian digunakan untuk mengurangi saldo dengan memeriksa nominal dan saldo yang tersedia.

```
package pekan1;

public class Rekening {
        String nomorRekening;
        String namaPemilik;
        double saldo;

        public Rekening(String nomor, String nama, double saldoAwal) {
                nomorRekening = nomor;
                namaPemilik = nama;
                saldo = saldoAwal;
                System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat
dengan saldo Rp" + saldo);
                System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
        }

        public void setorTunai(double nominal) {
                if (nominal > 0) {
                        saldo += nominal;
                        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo
saat ini: Rp" + saldo);
                } else {
                        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
                }
        }

        public void cekInformasi() {
                System.out.println("--- INFO REKENING ---");
                System.out.println("No. Rekening : " + nomorRekening);
                System.out.println("Nama Pemilik : " + namaPemilik);
                System.out.println("Saldo Akhir  : Rp" + saldo);
```

```
                System.out.println("---------------------");
        }

        public void tarikTunai(double nominal) {
                if (nominal <= 10000) {
                        System.out.println("Transaksi Gagal: Minimal nominal penarikan
10.000");
                } else if (nominal > saldo) {
                        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp." + saldo);
                } else {
                        System.out.println("Tarik tunai Rp" + nominal + " berhasil");
                        System.out.println("Saldo anda sekarang: Rp." + saldo);
                }
        }
}
```

Program dijalankan melalui class *Main*. *Scanner* membaca masukan pengguna, perulangan *while* menjaga menu tetap berjalan, dan *switch* memproses pilihan menu. Variabel akunAktif menunjukkan rekening yang sedang digunakan. Ketika kebutuhan program berkembang menjadi beberapa rekening, daftar rekening disimpan menggunakan ArrayList dan menu Ganti Akun digunakan untuk memilih rekening aktif. Dengan dasar ini, fitur pada modul berikutnya dapat dikembangkan di atas objek rekening yang sama.

```
package pekan1;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {

        public static void main(String[] args) {
                Scanner input = new Scanner(System.in);
                ArrayList<Rekening> daftarRekening = new ArrayList<>(); // Menyimpan semua
rekening
                Rekening akunAktif = null; // Objek belum dinisialisasi (null)
                boolean isRunning = true;
                 System.out.println("=== SISTEM PERBANKAN MINI ===");

                 while (isRunning) {
                        System.out.println("\nMenu Utama:");
                        System.out.println("1. Buka Rekening Baru");
                        System.out.println("2. Setor Tunai");
                        System.out.println("3. Tarik Tunai");
                        System.out.println("4. Cek Informasi Rekening");
                        System.out.println("5. Ganti Akun");
                        System.out.println("0. Keluar");
                        System.out.println("Pilih menu: ");

                        int pilihan = input.nextInt();
                        input.nextLine(); // Membersihkan buffer enter
```

```
                        switch(pilihan) {
                                case 1:
                                        System.out.print("Masukkan No Rekening: ");
                                        String no = input.nextLine();
                                        System.out.print("Masukkan Nama Pemilik: ");
                                        String nama = input.nextLine();
                                        System.out.print("Masukkan Saldo Awal: ");
                                        double saldo = input.nextDouble();
                                        input.nextLine();

                                        // Instansiasi object / Menjalankan constructor
                                        Rekening rekeningBaru = new Rekening(no, nama,
saldo);
                                        daftarRekening.add(rekeningBaru); // Tambahkan ke
ArrayList, tidak menimpa yang lama
                                        akunAktif= rekeningBaru; // Otomatis jadi akun aktif
setelah dibuat
                                        break;
                                case 2:
                                        if (akunAktif == null) {
                                                System.out.println("Error : Mohon maaf,
Anda belum memiliki nomor rekening");
                                        } else {
                                                System.out.print("Masukkan nominal setor:
");

                                                double setor = input.nextDouble();
                                                akunAktif.setorTunai(setor); // Memanggil
Behavior / method
                                        }
                                        break;
                                case 3:
                                        if (akunAktif == null) {
                                                System.out.println("Error : Mohon maaf,
Anda belum memiliki nomor rekening");
                                        } else {
                                                System.out.print("Masukkan nominal tarik:
");

                                                double tarik = input.nextDouble();
                                                akunAktif.tarikTunai(tarik);
                                        }
                                        break;
                                case 4:
                                        if (akunAktif == null) {
                                                System.out.println("Error: Anda belum
membuka rekening!");
                                        } else {
                                                akunAktif.cekInformasi();
                                        }
                                        break;
                                case 5:
                                        if (daftarRekening.isEmpty()) {
                                                System.out.println("Error : Belum ada
rekening yang terdaftar!");
                                        } else {
                                                System.out.print("Masukkan No Rekening yang
ingin diaktifkan: ");

                                                String cariNo = input.nextLine();

                                                Rekening ditemukan = null;
                                                for (Rekening r : daftarRekening) {
                                                        if (r.nomorRekening.equals(cariNo))
{
                                                                ditemukan = r;
```

```
                                                                        break;
                                                                }
                                                        }

                                                        if (ditemukan != null) {
                                                                akunAktif = ditemukan;
                                                                System.out.println("Berhasil
beralih ke akun atas nama " + akunAktif.namaPemilik);
                                                        } else {
                                                                System.out.println("Error : Nomor
rekening tidak ditemukan!");
                                                        }
                                                }
                                                break;
                                        case 0:
                                                isRunning = false;
                                                System.out.println("Sistem ditutup. Terima kasih!");
                                                break;
                                        default:
                                                System.out.println("Pilihan tidak valid!");
                                }
                        }
                        input.close();
                }

}
```

2. Modul 2: Relasi Antar Objek

Setelah proses rekening dan transaksi dasar tersedia, program dikembangkan agar setiap rekening dapat menyimpan riwayat transaksinya. *Class* Transaksi menyimpan ID transaksi, jenis transaksi, dan nominal.

```
package pekan2;

public class Transaksi {

        String idTransaksi;
        String jenis;
        double nominal;

        // Constructor
        public Transaksi(String id, String jenis, double nominal) {
                this.idTransaksi = id;
                this.jenis = jenis;
                this.nominal = nominal;
        }

        public void cetakDetail() {
                System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal:
Rp" + nominal);
        }
}
```

Pada *class* Rekening, ArrayList digunakan untuk menampung kumpulan objek Transaksi sehingga satu rekening dapat memiliki lebih dari satu catatan transaksi.

```
        // Implementasi Asosiasi (1-to-many)
        ArrayList<Transaksi> riwayatTransaksi;

        public Rekening2(String nomor, String nama, double saldoAwal) {
                nomorRekening = nomor;
                namaPemilik = nama;
                saldo = saldoAwal;
                System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat
dengan saldo Rp" + saldo);

                // Wajib menginisialisasi ArrayList di dalam constructor agar tidak
NullPointerException
                this.riwayatTransaksi = new ArrayList<>();

                System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
        }
```

Setoran tunai dicatat sebagai Kredit dengan ID berawalan TRX-S-. Penarikan tunai dicatat sebagai Debit dengan ID berawalan TRX-T-, tetapi hanya setelah penarikan berhasil.

```
        public void setorTunai(double nominal) {
                if (nominal > 0) {
                        saldo += nominal;
                        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
                        String idTrx = "TRX-S-" + System.currentTimeMillis();
                        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
                        riwayatTransaksi.add(trxBaru);

                        System.out.println("Setor tunai Rp." + nominal + " berhasil. Saldo
saat ini: Rp" + saldo);
                } else {
                        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
                }
        }
```

```
public void tarikTunai(double nominal) {
                if (nominal <= 10000) {
                        System.out.println("Transaksi Gagal: Minimal nominal penarikan
10.000");
                } else if (nominal > saldo) {
                        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp." + saldo);
                } else {
                        saldo = saldo - nominal;
                        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
                        String idTrx = "TRX-T-" + System.currentTimeMillis();
                        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
                        riwayatTransaksi.add(trxBaru);

                        System.out.println("Tarik tunai Rp." + nominal + " berhasil");
                        System.out.println("Saldo anda sekarang: Rp." + saldo);
```

```
                }
        }
```

Method cetakMutasi() menampilkan catatan transaksi menggunakan perulangan *for-each*. Jika daftar transaksi masih kosong, program menampilkan pesan bahwa belum ada transaksi pada rekening tersebut. Penambahan ini membuat perubahan saldo memiliki riwayat yang dapat diperiksa.

```
public void cetakMutasi() {
                System.out.println("=== MUTASI REKENING " + nomorRekening + " ===");
                if (riwayatTransaksi.isEmpty()) {
                        System.out.println("Belum ada transaksi pada rekening ini.");
                } else {
                        for (Transaksi trx : riwayatTransaksi) {
                                trx.cetakDetail();
                        }
                }
        }
```

3. Modul 3: Enkapsulasi

Pada tahap berikutnya, atribut yang perlu dilindungi diatur menggunakan *access modifier private*.

```
// 1. Mengunci atribut dengan 'private'
        private String nomorRekening;
        private String namaPemilik;
        private double saldo;
        private String pin; // Data sensitif!
```

Data seperti nomor rekening, nama pemilik, dan PIN tidak diakses langsung dari luar class. Akses terhadap data tertentu dilakukan melalui *method* yang disediakan, misalnya getter untuk membaca nomor rekening dan nama pemilik.

```
// 3. Getter untuk atribut yang diizinkan dibaca publik
        public String getNomorRekening() { return nomorRekening; }
        public String getNamaPemilik() { return namaPemilik; }
```

Constructor menerima PIN saat rekening dibuat. Jika panjang PIN bukan enam karakter, program menggunakan PIN default 123456. *Method* otentikasi() memeriksa PIN yang dimasukkan dan menghitung percobaan yang gagal.

```
// 2. Modifikasi Constructor untuk menerima PIN awal
        public Rekening3(String nomor, String nama, double saldoAwal, String pinAwal) {
                this.nomorRekening = nomor;
                this.namaPemilik = nama;
                this.saldo = saldoAwal;

                // Validasi PIN di dalam Constructor
                if (pinAwal.length() == 6) {
```

```
                        this.pin = pinAwal;
                } else {
                        System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN
default 123456");
                        this.pin = "123456";
                }

                this.riwayatTransaksi = new ArrayList<>();
                System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat
dengan saldo " + saldo);
        }
```

Setelah tiga kali kesalahan, rekening ditandai terblokir.

```
// percobaan PIN salah & status blokir
        private int percobaanGagal = 0;
        private boolean terblokir = false;
        private static final int BATAS_PERCOBAAN = 3;

// getter status blokir, hanya bisa dibaca dari luar, tidak bisa diubah langsung ---
        public boolean isTerblokir() { return terblokir; }

        // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
        public boolean otentikasi(String inputPin) {
                if (terblokir) {
                        return false; // Sudah terblokir, otentikasi selalu gagal
                }

                if (this.pin.equals(inputPin)) {
                        percobaanGagal = 0; // reset hitungan kalau PIN benar
                        return true;
                } else {
                        percobaanGagal++;
                        if (percobaanGagal >= BATAS_PERCOBAAN) {
                                terblokir = true;
                        }
                        return false;
                }
        }
```

Pemeriksaan PIN digunakan sebelum penarikan tunai dan pencetakan mutasi, sehingga kedua fitur tersebut terhubung dengan mekanisme keamanan rekening yang sama.

4. Modul 4: Ekspansi Produk Keuangan (Inheritance)

Setelah class rekening dasar memiliki data, riwayat, dan autentikasi, class tersebut dikembangkan menjadi beberapa jenis rekening melalui inheritance. RekeningTabungan dan RekeningGiro mewarisi fitur dasar rekening, sedangkan masing-masing memiliki atribut dan perilaku tambahan. *Constructor subclass* menggunakan super() untuk menjalankan *constructor class* induk.

```
//super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
                super(nomor, nama, saldoAwal, pinAwal);
```

RekeningTabungan memiliki atribut sukuBunga dan *method* tambahBungaAkhirBulan() untuk menghitung bunga dari saldo lalu menambahkannya ke rekening. Catatan bunga disimpan dalam riwayat dengan jenis Bunga dan ID berawalan TRX-B-.

```
// Atribut spesifik yang hanya dimiliki oleh Tabungan
        private double sukuBunga;
```

```
public void tambahBungaAkhirBulan() {
                // Menghitung bunga
                // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
                double nominalBunga = saldo * (sukuBunga / 100);
                saldo += nominalBunga;

                // Mencatat riwayat transaksi
                String idTrx = "TRX-B-" + System.currentTimeMillis();
                riwayatTransaksi.add(new Transaksi3(idTrx, "Bunga", nominalBunga));

                System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" +
nominalBunga);
        }
```

RekeningGiro memiliki batasOverdraft yang memungkinkan penarikan melampaui saldo selama nominalnya tidak melebihi saldo ditambah batas *overdraft*.

```
private double batasOverdraft;
```

```
        // Getter khusus Giro
        public double getBatasOverdraft() {
                return batasOverdraft;
        }
```

Upcasting memungkinkan objek berbagai jenis rekening disimpan dalam satu daftar bertipe *class* induk. Pemeriksaan *instanceof* dan *downcasting* digunakan saat program perlu menjalankan fitur khusus seperti simulasi bunga tabungan.

```
                                        if (produk == 1) {
                                                System.out.print("Masukkan suku bunga (%):
");

                                                double sukuBunga = input.nextDouble();
                                                input.nextLine();
                                                // Upcasting: RekeningTabungan disimpan di
variabel Rekening4

                                                rekeningBaru = new RekeningTabungan(no,
nama, saldo, pinBaru, sukuBunga);
                                        } else if (produk == 2) {
                                                System.out.print("Masukkan batas overdraft:
");

                                                double batasOverdraft = input.nextDouble();
                                                input.nextLine();
                                               // Upcasting: RekeningGiro disimpan di
variabel Rekening4
```

```
                                        rekeningBaru = new RekeningGiro(no, nama,
saldo, pinBaru, batasOverdraft);
                                    } else {
                                        System.out.println("Pilihan produk tidak
valid! Pembukaan rekening dibatalkan.");
                                        break;
                                    }
```

5. Modul 5: Transaksi Dinamis (Polymorphism)

Pada modul terakhir, *method* tarikTunai() pada *class* Rekening diubah agar mengembalikan boolean.

```
// Mengubah void menjadi boolean
        public boolean tarikTunai(double nominal) {
                if (terblokir) {
                        // akun terblokir
                        System.out.println("Akun anda terblokir");
                        return false;
                }
```

Nilai *true* menunjukkan penarikan berhasil, sedangkan *false* menunjukkan penarikan gagal.

```
if (nominal <= 0) {
                        System.out.println("Transaksi Gagal: Nominal tarik harus lebih dari
0!");
                        return false;
                } else if (nominal > saldo) {
                        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp." + saldo);
                        return false;
                } else {
                        saldo -= nominal;
                        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
                        String idTrx = "TRX-T-" + System.currentTimeMillis();
                        Transaksi4 trxBaru = new Transaksi4(idTrx, "Debit", nominal);
                        riwayatTransaksi.add(trxBaru);

                        System.out.println("Tarik tunai Rp." + nominal + " berhasil");
                        System.out.println("Saldo anda sekarang: Rp." + saldo);
                        return true; // Transaksi sukses
                }
        }
```

RekeningGiro melakukan *overriding* pada method tersebut untuk menerapkan aturan *overdraft*. Dengan demikian, ketika penarikan dipanggil melalui referensi bertipe Rekening, implementasi yang dijalankan menyesuaikan objek rekening yang digunakan.

```
// IMPLEMENTASI POLIMORFISME (OVERRIDING)
        @Override
        public boolean tarikTunai(double nominal) {
                if (nominal <= 0) {
```

```
                        System.out.println("Transaksi Gagal: Nominal tarik harus lebih dari
0!");
                        return false;
                }
                // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi (saldo +
batasOverdraft)
                else if (nominal > (saldo + batasOverdraft)) {
                        System.out.println("Transaksi Gagal: Melebihi batas overdraft!
Maksimalkan penarikan: Rp" + (saldo + batasOverdraft));
                        return false;
                }
                else {
                        saldo -= nominal; // Saldo bisa bernilai negatif di sini (memakan
overdraft)

                        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
                        String idTrx = "TRX-T-" + System.currentTimeMillis();
                        Transaksi4 trxBaru = new Transaksi4(idTrx, "Debit", nominal);
                        riwayatTransaksi.add(trxBaru);

                        System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil.");
                        System.out.println("Saldo anda sekarang: Rp." + saldo);
                        return true; // Transaksi sukses
                }
        }
```

Program juga dilengkapi *method* transfer() untuk memindahkan nominal dari rekening asal ke rekening tujuan. Sebelum proses dilanjutkan, program memeriksa status blokir dan mengautentikasi PIN. Nominal transfer harus lebih dari nol dan rekening tujuan tidak boleh terblokir. Selanjutnya, program memanggil tarikTunai() pada rekening asal. Saldo rekening tujuan ditambah hanya jika penarikan tersebut berhasil. Karena pemanggilan tarikTunai() menggunakan method yang dapat di *override*, proses transfer juga mengikuti aturan penarikan dari jenis rekening yang sedang aktif.

```
public void transfer(Rekening5 tujuan, double nominal, String pinInput) {
            if (terblokir) {
                System.out.println("Akun anda terblokir");
                return;
            }
            if (!otentikasi(pinInput)) {
                if (isTerblokir()) {
                    System.out.println("Akun anda terblokir");
                } else {
                    System.out.println("Akses ditolak: PIN salah!");
                }
                return;
            }
            if (nominal <= 0) {
                System.out.println("Transfer gagal: Nominal harus lebih dari 0!");
                return;
            }
            if (tujuan.isTerblokir()) {
                System.out.println("Transfer gagal: Rekening tujuan terblokir!");
                return;
            }
```

```
            boolean berhasil = this.tarikTunai(nominal);

            if (berhasil) {
                tujuan.setorTunai(nominal);
                System.out.println("\nTransfer berhasil!");
                System.out.println("Rekening tujuan: " + tujuan.getNomorRekening());
                System.out.println("Nominal transfer: Rp." + nominal);
                        System.out.println("Saldo anda sekarang: Rp." + saldo);

            } else {
                System.out.println("Transfer gagal!");
            }
        }
```

Fitur transfer diintegrasikan ke dalam menu utama. Pengguna memasukkan nomor rekening tujuan, kemudian program mencari rekening tersebut dalam daftar rekening yang tersedia. Jika rekening tujuan ditemukan, pengguna dapat memasukkan nominal transfer dan PIN untuk melanjutkan transaksi.

```
System.out.println("8. Transfer");
```

```
                        case 8:
                                if (akunAktif == null) {
                                        System.out.println("Error: Anda belum membuka
rekening!");
                                } else if (akunAktif.isTerblokir()) {
                                        System.out.println("Akun anda terblokir");
                                } else {
                                        System.out.print("Masukkan No Rekening Tujuan: ");
                                        String noTujuan = input.nextLine();

                                        Rekening5 akunTujuan = null;

                                        for (Rekening5 r : daftarRekening) {
                                                if (r.getNomorRekening().equals(noTujuan)) {
                                                        akunTujuan = r;
                                                        break;
                                                }
                                        }
                                        if (akunTujuan.isTerblokir()) {
                                                System.out.println("Error: Rekening tujuan
terblokir!");
                                        } else {
                                                System.out.print("Masukkan nominal transfer:
");
                                                double nominal = input.nextDouble();
                                                input.nextLine();

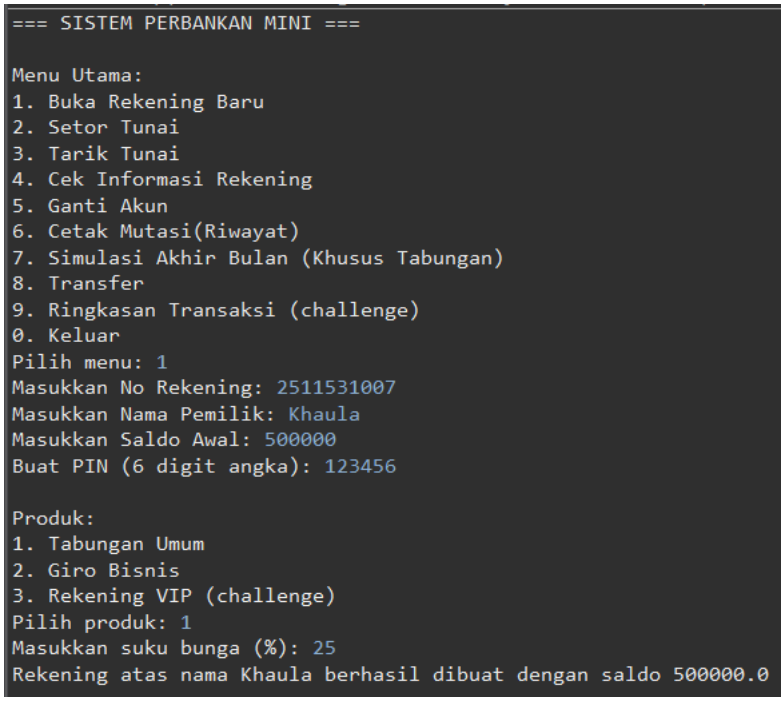
                                                System.out.print("Masukkan PIN: ");
                                                String pin = input.nextLine();

                                                akunAktif.transfer(akunTujuan, nominal,
pin);
                                        }
                                }
                                break;
```

B. HASIL KODE PROGRAM

1. Buka Rekening Baru

Menampilkan proses pembuatan rekening baru dengan memasukkan data yang diperlukan, termasuk jenis rekening.



2. Setor Tunai

Menampilkan proses penyetoran uang ke rekening dan perubahan saldo setelah transaksi berhasil dilakukan.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 35000
Setor tunai Rp.35000.0 berhasil. Saldo saat ini: Rp535000.0
```

3. Tarik Tunai

Menampilkan proses penarikan uang dari rekening. Saldo akan berkurang jika penarikan berhasil dilakukan sesuai ketentuan program.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 250000
Tarik tunai Rp.250000.0 berhasil
Saldo anda sekarang: Rp.285000.0
```

4. Cek Informasi Rekening

Menampilkan informasi rekening yang sedang aktif, seperti nomor rekening, nama pemilik, dan saldo.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 2511531007
Nama Pemilik : Khaula
Saldo Akhir  : Rp.285000.0
Status       : Aktif
--------------------
```

5. Ganti Akun

Menampilkan proses pemilihan rekening lain dari daftar rekening yang tersedia. Setelah akun diganti, program menggunakan rekening yang telah dipilih.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 5
Masukkan No Rekening yang ingin diaktifkan: 2511531001
Berhasil beralih ke akun atas nama Dzhillan
```

6. Cetak Mutasi (Riwayat)

Menampilkan riwayat transaksi pada rekening aktif, seperti setor tunai, tarik tunai, dan transaksi lainnya yang tercatat.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 6
Masukkan PIN: 123456
=== MUTASI REKENING 2511531007 ===
ID: TRX-S-1793021786851 | Jenis: Kredit | Nominal: Rp35000.0
ID: TRX-T-1793021822617 | Jenis: Debit | Nominal: Rp250000.0
```

7. Simulasi Akhir Bulan (Khusus Tabungan)

Menampilkan proses simulasi akhir bulan untuk menghitung dan menambahkan bunga pada saldo rekening tabungan.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 7
Bunga 25.0% berhasil ditambahkan: Rp71250.0
```

8. Transfer

Menampilkan proses transfer uang dari rekening aktif ke rekening tujuan. Jika transfer berhasil, saldo rekening asal berkurang dan saldo rekening tujuan bertambah.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
9. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 8
Masukkan No Rekening Tujuan: 2511531001
Masukkan nominal transfer: 250000
Masukkan PIN: 123456
Tarik tunai Rp.250000.0 berhasil
Saldo anda sekarang: Rp.106250.0
Setor tunai Rp.250000.0 berhasil. Saldo saat ini: Rp400000.0
```

D. KESIMPULAN

Praktikum Modul 1 sampai 5 membangun Sistem Perbankan Mini sedikit demi sedikit, dan setiap modul menambahkan satu konsep PBO ke program yang sama. Di Modul 1, program bisa membuat rekening, setor tunai, tarik tunai, dan menampilkan informasi rekening. Modul 2 menambahkan *class* Transaksi dan ArrayList untuk menyimpan riwayat transaksi, sehingga fitur cetak mutasi dapat menampilkan transaksi yang sudah dilakukan.

Modul 3 membatasi akses atribut dengan *encapsulation*. Program juga mulai meminta PIN, dan rekening diblokir setelah tiga kali gagal autentikasi. Di Modul 4, RekeningTabungan dan RekeningGiro mewarisi *class* Rekening, lalu masing-masing diberi fitur sendiri: bunga untuk tabungan dan batas overdraft untuk giro. Modul 5 memakai *polymorphism* lewat *overriding* method tarikTunai() dan menambahkan transfer antar rekening, dengan pemeriksaan sebelum dana dipindahkan.

Dengan demikian, program yang awalnya hanya mengelola rekening dasar kini memiliki fitur yang lebih beragam dan terstruktur, sehingga praktikum ini membantu memahami penerapan konsep PBO.

E. LINK REPOSITORY GITHUB

Repository GitHub praktikum:

https://github.com/khaulaathifa2/PrakPBO_C_2511531007

F. LINK VIDEO PENJELASAN

Link youtube:

https://youtu.be/sYBPFU2IeSA?si=upYqLOP8iONZ9RXC